

Python Code

```
1  """
2  Stock market prediction using Markov chains.
3
4  For each function, replace the return statement with your code. Add
5  whatever helper functions you deem necessary.
6  """
7
8  import comp140_module3 as stocks
9  import random
10
11  ### Model
12
13  def markov_chain(data, order):
14      """
15      Create a Markov chain with the given order from the given data.
16
17      inputs:
18          - data: a list of ints or floats representing previously collected data,
19            where data is >0
20          - order: an integer representing the desired order of the markov chain,
21            where 0<= order < data
22
23      returns: a dictionary that represents the Markov chain
24      """
25      # markov chain
26
27      # initialize
28      markov = {}
29      total_transitions = len(data) - order
30      tuple_count = {}
31
32      # count number of states for each state then the frequency of the following
33      # numbers
34      for idx in range(total_transitions):
35          state = tuple([data[idx + jdx] for jdx in range(order)])
36          prediction = data[idx + order]
37          if state not in tuple_count:
38              tuple_count[state] = {}
39          tuple_count[state][prediction] = tuple_count[state].get(prediction, 0) + 1
40      # convert to probabilities ie order-order markov chain
41      for state, prediction_counts in tuple_count.items():
42          total = sum(prediction_counts.values())
43          markov[state] = {prediction: count / total for prediction,
44                           count in prediction_counts.items()}
45
46      return markov
47
48  ### Predict
49
```

```

50
51 def weighted_choice(prob_dict):
52     """
53     prob_dict: {value: probability} where probabilities sum to ~1
54     returns one value, chosen randomly according to the probabilities
55     """
56     # number in [0, 1)
57     rand = random.random()
58     cumulative = 0
59
60     # check to see if random falls into which probability 'bucket'
61     for value in prob_dict:
62         cumulative += prob_dict[value]
63         if rand < cumulative:
64             return value
65         # in case rand is still higher than cumulative it is just the last number
66         last_value = value
67     return last_value
68
69 def predict(model, last, num):
70     """
71     Predict the next num values given the model and the last values.
72
73     inputs:
74         - model: a dictionary representing a Markov chain
75         - last: a list (with length of the order of the Markov chain)
76               representing the previous states
77         - num: an integer representing the number of desired future states
78
79     returns: a list of integers that are the next num states
80     """
81     # initialize
82     predictions = []
83
84     # iterate num times
85     for dummy_idx in range(num):
86         # current state
87         state = tuple(last)
88
89         # state is not in model --> uniform distribution
90         if state not in model:
91             next_value = random.randint(0, 3)
92         else:
93             next_value = weighted_choice(model[state])
94         predictions.append(next_value)
95
96         # update last to represent previous states
97         last = last[1:] + [next_value]
98
99     return predictions
100
101
102 ### Error
103

```

```
104 def mse(result, expected):
105     """
106     Calculate the mean squared error between two data sets.
107
108     The length of the inputs, result and expected, must be the same.
109
110     inputs:
111         - result: a list of integers or floats representing the actual output
112         - expected: a list of integers or floats representing the predicted output
113
114     returns: a float that is the mean squared error between the two data sets
115     """
116     # simple mse formula implementation
117     total = 0
118     for index in range(len(result)):
119         diff = result[index] - expected[index]
120         total += diff ** 2
121
122     return total / len(result)
123
124
125 ### Experiment
126
127 def run_experiment(train, order, test, future, actual, trials):
128     """
129     Run an experiment to predict the future of the test
130     data given the training data.
131
132     inputs:
133         - train: a list of integers representing past stock price data
134         - order: an integer representing the order of the markov chain
135               that will be used
136         - test: a list of integers of length "order" representing past
137               stock price data (different time period than "train")
138         - future: an integer representing the number of future days to
139               predict
140         - actual: a list representing the actual results for the next
141               "future" days
142         - trials: an integer representing the number of trials to run
143
144     returns: a float that is the mean squared error over the number of trials
145     """
146     # initialize
147     markov = markov_chain(train, order)
148     total_error = 0
149
150     # run each of the trials trials
151     for dummy_trial in range(trials):
152         # pass a copy of test to prevent mutation
153         prediction = predict(markov, test[:], future)
154         total_error += mse(prediction, actual)
155     return total_error / trials
156
157
```

```
158     ### Application
159
160     def run():
161         """
162         Run application.
163
164         You do not need to modify any code in this function. You should
165         feel free to look it over and understand it, though.
166         """
167         # Get the supported stock symbols
168         symbols = stocks.get_supported_symbols()
169
170         # Get stock data and process it
171
172         # Training data
173         changes = {}
174         bins = {}
175         for symbol in symbols:
176             prices = stocks.get_historical_prices(symbol)
177             changes[symbol] = stocks.compute_daily_change(prices)
178             bins[symbol] = stocks.bin_daily_changes(changes[symbol])
179
180         # Test data
181         testchanges = {}
182         testbins = {}
183         for symbol in symbols:
184             testprices = stocks.get_test_prices(symbol)
185             testchanges[symbol] = stocks.compute_daily_change(testprices)
186             testbins[symbol] = stocks.bin_daily_changes(testchanges[symbol])
187
188         # Display data
189         # Comment these 2 lines out if you don't want to see the plots
190         stocks.plot_daily_change(changes)
191         stocks.plot_bin_histogram(bins)
192
193         # Run experiments
194         orders = [1, 3, 5, 7, 9]
195         ntrials = 500
196         days = 5
197
198         for symbol in symbols:
199             print(symbol)
200             print("====")
201             print("Actual:", testbins[symbol][-days:])
202             for order in orders:
203                 error = run_experiment(bins[symbol], order,
204                                       testbins[symbol][-order-days:-days], days,
205                                       testbins[symbol][-days:], ntrials)
206                 print("Order", order, ":", error)
207             print()
208
209         # You might want to comment out the call to run while you are
210         # developing your code. Uncomment it when you are ready to run your
211         # code on the provided data.
```

```
212 run()  
213
```